Speconsense: a brief history and a guided tour of the 0.8.0 pipeline

Josh Walker

Internal review draft, May 2026 (commit 896ed16)

Abstract

Speconsense is a per-specimen consensus tool for Oxford Nanopore amplicon reads, developed for the fungal DNA barcoding workflow that previously used NGSpeciesID [6], with automatic variant phasing as a first-class feature rather than a post-hoc step. The tool has been shaped over roughly seven months of production use at Mycota Lab (since November 2025), and the 0.8.0 release reorganizes the read-to-consensus path into thirteen sequential phases (six new in 0.8.0) that address three persistent concerns from the validator community: aggressive homopolymer normalization that hides real variation at homopolymer interfaces, ambiguity-code inflation in low-quality clusters, and the lack of a defensible test for low-support variants. Against the validator-facing pass track on a 955-specimen production run, 0.8.0 produces 27% more distinct candidate biological variants with 44% fewer ambiguity codes and a 5% yield gain over 0.7. Pass-track consensuses are also cleaner at the homopolymer interfaces where 0.7 most often hid real variation: the share of pass-track clusters with unphased homopolymer-interface heterogeneity at length-2-to-5 HP boundaries drops 8-fold (27% → 3%). Variants the new significance test flags as statistically indistinguishable from basecaller noise are routed to a separate inspection track rather than emitted to the main output. The routing thresholds are deliberate first cuts; an 0.8.1 fast-follow tightens the MAD outlier-removal default (`--mad-z-threshold` 3.0 → 1.5; §10), and a 0.8.2 revision will revisit the size-based thresholds based on partner review and user-acceptance testing. This paper is a guided tour of the pipeline as it stands, told through the development decisions that produced it, with empirical results from a per-phase additive ladder and cross-tool comparisons. The companion technical papers — variant significance [1], homopolymer error rate [2], and CER in practice [3] — provide the mathematical foundations.

1 Origins

Speconsense exists because a typical fungal ITS specimen is more interesting than a single sequence. A real specimen often contains genuine intragenomic variants, occasional contaminants, and the usual nanopore noise floor — and a barcoding pipeline that emits one consensus per specimen will silently average across all of them. The Russell barcoding protocol [5] used NGSpeciesID [6], which clusters reads but stops short of phasing variants and can fold biologically distinct sequences into a single output when their cluster shapes happen to overlap.

Initial development began in February 2025 with two concrete goals: (1) cluster reads in a way that could discover variants without being told how many to expect, and (2) generate a separate consensus for each variant, with all the bookkeeping needed to trace each output back to its supporting reads. After a small handful of commits in February and March, the project went on hiatus for roughly five months. It resumed in late August 2025 when Harte Singer at the Fungal Diversity Survey (FUNDIS) raised the need for SNP calling at scale across a large fungal-barcoding dataset; the author proposed speconsense as a substitute for the NGSpeciesID-based step in the existing workflow, and serious development began that week. A guiding principle,

articulated explicitly later, was *split rather than conflate* — when in doubt, emit two clusters and let the post-processing tool merge them, rather than collapse them in core and lose the distinction forever.

2 Choosing MCL

The dominant question for the clustering step was: how do we cluster reads when we don't know in advance how many clusters there should be? K-means and similar partition-based methods require the cluster count up front. Hierarchical clustering produces a dendrogram but pushes the cut-height decision onto the user. Greedy clustering as a class spans many implementations — some, like `vsearch --cluster_fast`, use a k-mer index and scale gracefully. The straightforward greedy variant that speconsense initially considered, however, tends to produce a single dominant cluster plus a long tail of fragments, and (as we will see in Section 2.1) requires a dense pairwise comparison matrix that becomes prohibitive at scale.

Markov Clustering (MCL) [8] was chosen as the default because it sits in the right part of the design space: it operates on a similarity graph, it discovers the cluster count from the graph's structure rather than as a parameter, and its inflation parameter gives a single intuitive knob for cluster granularity. Importantly, MCL operates on a *sparse* graph — typically only the strongest edges from each read are retained — which scales gracefully to tens of thousands of reads. A greedy algorithm (speconsense's own implementation, working from the same dense pairwise matrix MCL builds) is retained as a fallback when MCL is not installed, and as an option for users whose primary use case is contaminant detection rather than variant resolution.

2.1 Building the similarity graph

MCL needs a graph as input, and the graph is what determines the clustering. Speconsense builds it by comparing every read against every other read using pairwise sequence alignment — the same kind of comparison familiar from BLAST output, where two sequences are scored by their percent identity over their alignable region. There is one critical caveat for readers used to thinking about ITS identity thresholds at the OTU or species level: those thresholds (commonly cited at 99.0–99.5% for "same OTU" and around 95% for "related but distinct") are calibrated for *consensus* sequences, where nanopore basecalling error has already been averaged out. The comparisons here are between *raw reads*, each carrying its own basecaller error, so two reads from the same biological sequence routinely score in the 90s rather than at 99%+. The graph is built in raw-read identity space; the OTU thresholds are not the right yardstick for it.

In a specimen with a few hundred reads this exhaustive comparison is cheap; in a specimen with tens of thousands of reads it is the dominant cost in the pipeline, and Section 5 describes how 0.7 made it tractable at scale. The output is a similarity matrix: read i vs read j has score s_{ij}.

The matrix is then sparsified into a graph. For each read, only the edges to its few most-similar neighbors are retained — this is the "K-nearest-neighbors" step, with K controlled by `--k-nearest-neighbors` (default 5). Keeping only the strongest edges does two useful things at once: it removes noise edges between weakly-related reads that would otherwise pull MCL toward over-merging, and it prevents the graph from becoming so dense that MCL's matrix operations slow to a crawl. The remaining edges, weighted by their identity scores, form the graph that MCL clusters.

The reason this graph structure works for variant discovery is mechanical: real biological variants in a specimen produce dense subgraphs (each read is highly similar to other reads from the same variant, and a few hops away from reads of a different variant), while sequencing noise produces

a thin scatter of edges with no internal structure. MCL is good at finding the dense subgraphs and ignoring the scatter. The initial proof of concept was strikingly good at this from the start — minor sequence variants surfaced as separate clean clusters even though MCL was only ever shown a sparse K-NN slice of the pairwise identity matrix. That early result was what convinced us the rest of the pipeline was worth building.

3 Variant phasing through MSA

Speconsense has always done a kind of variant phasing implicitly — whenever MCL splits a specimen's reads into multiple clusters, the resulting per-cluster consensuses already represent distinct variants. What was missing through the 0.4 and 0.5 series was *explicit* variant phasing: a mechanism to discover variation *within* a cluster and split it on a per-position basis. Position-based, MSA-driven phasing was prototyped in November 2025 (`8f18e8d` added variant position detection on November 9; `44ab13b` implemented automatic phasing on November 11) and shipped in v0.6.0 on December 5, 2025.

The approach is straightforward in outline: build a multiple sequence alignment (MSA) of the reads in a cluster using SPOA [10, 9], find columns where multiple bases each occur with non-trivial frequency, and split the reads by their pattern across those positions. The version that shipped in v0.6.0 uses a recursive hierarchical algorithm that picks the most informative position at each level and partitions reads, recursing into the resulting subsets until no further phaseable positions remain.

For positions that survive variant analysis but cannot cleanly partition the reads — typically because the minor allele appears at intermediate frequency without grouping with other minor alleles — the consensus emits an IUPAC ambiguity code (R for A/G, Y for C/T, etc.). Ambiguity calling was added in `306a00e` (December 1, 2025) and gave validators a way to see residual heterogeneity that the phaser couldn't resolve.

3.1 A note on terminology

We use "variant phasing" rather than "haplotype phasing" throughout. Many of the variants the pipeline finds in fungal ITS data are not haplotypes in the strict population-genetic sense: paralogous rDNA copies, intragenomic variation within a single individual, and other non-allelic differences appear regularly, and "variant" covers all of these without overcommitting to a model.

3.2 Two cluster-level metrics: `err_factor` and `cer_factor`

Two derived metrics recur throughout the rest of the paper. We introduce them informally here; precise definitions live in the companion papers [1, 3].

- `err_factor` is a per-cluster quality metric. It is the ratio of the disagreement actually observed among the reads of a cluster to the disagreement an empirical basecaller error model predicts for that cluster. A cluster of reads that are all faithful copies of a single biological sequence, differing only by basecaller noise, has `err_factor` ≈ 1. A cluster that is a mix of related-but-distinct sequences has `err_factor` much greater than 1, because the mix produces per-position disagreement above what the error model alone would explain. `err_factor` is *peer-independent*: it can be computed from a single cluster in isolation.

- `cer_factor` is a peer-relative significance metric drawn from the *critical error rate* (CER) framework [1, 3]: a per-variant significance test calibrated against the platform's actual error

rate, named for the per-position rate p^* at which an observed variant pattern would just become plausible as artifact. For a smaller cluster sitting near a larger peer in the same identity group, `cer_factor` asks: what per-position error rate would the basecaller need for the smaller cluster's variant pattern to be plausibly explained as basecaller artifact, expressed as a multiple of the basecaller's actual error rate? `cer_factor` much less than 1 means the actual basecaller rate is already higher than what would be needed — the variant pattern is well within what basecaller noise could produce, and is most likely noise. `cer_factor` much greater than 1 means the basecaller would need to err many times faster than it actually does to produce the observed pattern — the variant is implausible as basecaller artifact and is likely real biology. The dimensionless factor is p^*/q_{ctx} in the notation of [3]. We will refer to "CER" and "the significance framework" interchangeably from here on.

The two metrics answer two genuinely different validator questions: "is this cluster real?" (`err_factor`) and "is this variant distinguishable from its peer?" (`cer_factor`). The 0.8 pipeline computes and surfaces both, and uses each for the decision it is appropriate for.

4 From proof of concept to production: 0.4 through 0.6

Through the spring 2025 dormancy, the tool was a proof of concept — runnable but not used by anyone other than the author. Two events in the autumn brought it into active partnership. Following Harte Singer's August enquiry, Stephen Russell at Mycota Lab began testing speconsense seriously against production data in early October 2025. A detailed UX conversation in mid-October turned into a release one week later: **v0.4.0 (October 23, 2025)** introduced customizable FASTA header fields, a quality report aimed at flagging sequences worth a second look, MSA-based variant merging with `.raw` traceability files for full provenance, and incremental output writing for large datasets. The release addressed most of the items on Stephen's feedback list and marked the transition from proof-of-concept to **preview** — a stable enough surface for someone other than the author to run real workloads on it.

Through late October and November, Mycota Lab ran experimental batches with v0.4.x and v0.5.x. Two pieces of feedback from this period drove the next major release.

The first (October 30) was a feature request for primer-pool support. Mycota Lab's workflow amplifies a single specimen with multiple primer pairs that target overlapping but non-identical stretches of the same locus — in this case ITS1F/ITS4 (full ITS) and gITS7/ITS4 (ITS2 only, with the forward primer sitting in the 5.8S region). The two amplicons share the ITS2 + LSU end but differ in length, and the summarize tool needed to recognize when a ∼360 bp gITS7–ITS4 amplicon and a ∼660 bp ITS1F–ITS4 amplicon were "the same sequence" with respect to their shared region. Implementation was deferred to a fast-follow release, **v0.6.1** (Section 4.1).

The second (November 7) was a flag from Stephen that residual unphased heterogeneity was visible in some specimens — clear two-base patterns at certain positions across most of a cluster, but the consensus reporting only the majority. This aligned with MSA analysis work the author had begun the same week, and turned into the headline feature of the next major release. **v0.6.0 (December 5, 2025)** shipped automatic detection of variant positions, MSA-based variant phasing, and IUPAC ambiguity codes for positions that didn't cleanly resolve into separate variants. v0.6.0 was the first release used in production at Mycota Lab.

The principle behind the ambiguity-calling work was **always express uncertainty clearly**. If the reads disagree at a position and the disagreement cannot be cleanly resolved by phasing, the consensus should say so — by emitting an IUPAC ambiguity code (R for A/G, Y for C/T, etc.) rather than picking a winner. The previous behavior had let the consensus assert certainty (a

single base) where the underlying reads showed none, which validators reasonably read as the tool actively hiding information.

This was the right principle but the implementation was under-defended. Ambiguity calls were generated at any variant position that survived the frequency threshold but failed to phase, with no separate test for whether the cluster as a whole was clean enough for an ambiguity call to be meaningful. Validators eventually started seeing the symmetric problem: clusters with so many ambiguity codes that the consensus was unhelpful, because the cluster itself was a mix of related-but-distinct sequences that should have been split apart or filtered out. The principle was correct, but it needed noise-reduction machinery that did not arrive until 0.8.0 (Section 6). In the meantime, ambiguity codes did at least make the underlying issue visible, which is more than the previous behavior had done.

4.1 The chimeric merge issue (v0.6.1)

The primer-pool feature requested in October shipped in v0.6.1 (December 18, 2025) as overlap-aware identity scoring: the distance metric between two sequences would only score the region they had in common. This worked for the intended primer-pool case. It also created a chimeric-merge failure mode: two sequences from genuinely different organisms that happened to share enough overlap with a third sequence could merge transitively into a chimera. The fix, in `3f82c66`, made the overlap distance fire only between sequences with *different* primer pairs, restoring strict identity scoring within a primer pair. The current rule is still permissive: any two amplicons from different primer pairs are eligible to merge on their overlapping region. Future work will tighten this to merge only across primer pairs whose loci are *a priori* expected to overlap biologically (e.g., an ITS1F/ITS4 amplicon and a gITS7/ITS4 amplicon), refusing the merge when the primer-pool relationship does not predict biological co-occurrence of the shared region.

4.2 IUPAC conflation bugs

Two IUPAC bugs surfaced in the 0.6 series, with different origins but the same flavor.

The first (`ca3fc20`, v0.6.4) was a code-expansion bug in variant merging. When merging a base with an existing IUPAC code — say C against Y — the code naively unioned the literal symbols and produced N, when the correct result was Y (since Y already covers C and T). The fix added a helper that expanded existing codes to their constituent bases before re-encoding.

The second (`b83449f`, December 2025 / v0.7) was a defaults issue, not a code bug, but presented to users the same way. Summarize would merge variants and propagate IUPAC codes regardless of the relative sizes of the input clusters. A 3-read cluster with one minor SNP could merge into a 500-read cluster and inject an ambiguity code into an otherwise clean consensus. The validators' implicit expectation was that every *read* would count equally; the implementation gave every *cluster consensus* an equal vote, which is a different thing. The design rationale was not equal voting per se, but *preservation of information* — when a small cluster carried a SNP the dominant cluster did not, we wanted that difference visible in the merged consensus rather than silently dropped. The two intentions agree most of the time but diverge when cluster sizes are imbalanced, which is exactly when an ambiguity code in an otherwise-clean consensus surprises the reader. The fix raised the default `--merge-min-size-ratio` from 0.0 to 0.1, requiring a candidate to be at least 10% the size of the dominant cluster to participate in the merge.

5 Scalability and ergonomics: 0.7

The 0.7 series (v0.7.0 in January 2026 through v0.7.7 in March 2026) was the operational hardening release. The pieces:

- **Scalability mode.** Many of the pairwise operations in the pipeline are $O(n^2)$ by default, which is fine at hundreds of reads and painful at thousands. A vsearch-backed [12] candidate finder reduces the dominant operations to roughly $O(n \log n)$ by pruning the candidate set before edlib [11] scoring. Activates automatically above a configurable threshold.
- **Profile system.** YAML presets (`herbarium`, `strict`, `nostalgia`, `largedata`, `compressed`) expose a small set of curated parameter combinations matched to common workflows, with user-editable profiles in `~/.config/speconsense/profiles/`.
- **Threading.** A `--threads` option parallelizes consensus generation and clustering across cores.
- **Subpackage refactor.** The flat `core.py` and `summarize.py` were split into the current `core/` and `summarize/` subpackages, both for readability and to make selective testing tractable.

By the end of the 0.7 series the pipeline was stable and the main remaining concerns came from the validator community.

5.1 The 0.7 pipeline at a glance

For reference when reading the 0.8 description in Section 7, the 0.7 pipeline ran six phases per specimen:

Phase	Name	What it does
1	Initial clustering	MCL on a K-NN identity graph (greedy fallback)
2	Pre-phasing merge	Combine clusters with HP-equivalent consensuses
3	Variant phasing	MSA-based variant splitting; emit IUPAC codes for unphased positions
4	Post-phasing merge	Combine subclusters that became HP-equivalent after phasing
5	Size filtering	Drop clusters below `--min-size` and `--min-cluster-ratio`
6	Output generation	SPOA consensus, primer trimming, FASTA/FASTQ writeout

The summarize tool then ran identity grouping, MSA-based variant merging, optional cross-primer overlap merging, variant selection, and a quality report. Two architectural points to retain for the 0.8 comparison: (a) the levers available for variant selection were ad hoc — absolute size floor, ratio to the dominant cluster — with no statistical grounding, so a 5-read variant in a 300-read specimen and a 5-read variant in a 30-read specimen were treated identically despite the very different per-cluster significance; and (b) summarize made its own grouping decisions independent of core's clustering.

6 The three problems behind 0.8.0

Three classes of validator question kept recurring through the 0.7 series. Each one pointed at an architectural decision rather than a localized bug.

6.1 What to do about homopolymers

The most frequent question was a variant of: "I have a 50/50 mix of reads that look like CCCTT and CCTTT. The middle position looks like a clean SNP — why doesn't speconsense call an ambiguity here?" This was raised independently and over many specimens by Stephen Russell (Mycota Lab) and by Elora Adamson, Local Coordinator for the MycoMap BC Network project (British Columbia); Elora provided the most detailed cluster-level feedback and a running set of specimens that exhibited the issue.

The interface-homopolymer pattern was the recurring case study, but the underlying question is broader: what should the pipeline do about homopolymers *at all*? Through 0.7, speconsense applied homopolymer (HP) normalization whenever it compared two cluster *consensus* sequences — in pre- and post-phasing merges, identity-group formation, and summarize's variant merging. (Read-vs-read comparisons during initial similarity-matrix construction use raw edlib identity and have never been HP-normalized.) The mechanism in core is a native HP-aware scoring loop on top of an edlib alignment: within an HP run of qualifying length, an insertion or deletion of the run's base is not penalized, so a length-7 A-run and a length-8 A-run align as equivalent. The summarize tool uses the related `adjusted-identity` package [13] for the same purpose with a slightly different parameterization. In either implementation, runs of identical bases are not literally compressed to single characters, but the effective scoring is as if they were. Under that lens, CCCTT and CCTTT are equivalent — the only difference between them is the length of the C run and the length of the T run, and homopolymer length differences are the dominant systematic error mode in nanopore sequencing. The pipeline was designed to ignore them.

But the SNP interpretation is also reasonable. The middle position genuinely is C in some reads and T in others. A validator looking at a stack of reads doesn't see "two homopolymer runs of variable length" — they see a column where two bases appear at meaningful frequency, and ask why their tool isn't flagging it.

The right answer is empirical: how often is the SNP interpretation correct, and how often is it the homopolymer-noise interpretation? Paper 2 [2] addresses this directly. Briefly: the rate of artifactual variation at homopolymer-interface positions is no higher than at non-homopolymer positions (rate ratios of 1.03 and 0.88 for Dorado v3.5 and v5.0 respectively), so blanket normalization at short homopolymer lengths is too aggressive — it hides real variation behind a noise model that doesn't actually apply at those lengths. The fix is not to abandon HP normalization (it remains correct at long HP lengths, where the error rate genuinely is higher) but to make it context-aware: a per-event-type, per-HP-length error rate, applied as a continuous statistical weight rather than a hard switch.

Relaxing the normalization is mechanically straightforward but required new infrastructure: a context-aware error model (per-basecaller q_{ctx} tables [2], keyed by event type and homopolymer run length) so that variants at HP positions could be evaluated statistically rather than reflexively suppressed. We will refer to this empirical error model simply as the *error model* from this point on. The same model also drives the cluster-homogeneity metric `err_factor` (Phase 12) and the peer-relative `cer_factor` (Phase 10), so the work that started as "let validators see HP variation when it's real" ended up enabling the cluster-quality machinery as well.

6.2 Ambiguity inflation in low-quality clusters

The second recurring question was the inverse: "Why does this consensus have so many ambiguity codes? It looks like noise."

Investigation showed that some clusters were genuinely heterogeneous — a mix of reads from related but distinct sequences that the clustering step had grouped together because they were too close to separate at the configured identity threshold. Variant phasing would then attempt to split them, often fail to find a clean partition, and emit ambiguity codes at every disagreeing position. A 600 bp consensus with 30 ambiguity codes is unhelpful to a validator: it's neither a clean sequence nor a clear signal that the cluster should be discarded. There was no way for the tool to say "this cluster is not a real sequence, it is a noise blob" — the only filter was on size, which is fine for the easy cases but is a poor proxy when the goal also includes rescuing low-read specimens. A noise blob that survives the size floor is hard to flag without a distinct test for cluster cohesion.

6.3 Validating low-support variants

The third question was the hardest: "I have a 5-read cluster that's two SNPs different from a 200-read cluster. Is it a real variant?"

Through 0.7, the answer was effectively "we don't know — that's your call." Variants were filtered by absolute size and by ratio to the dominant cluster, and variants that survived both were emitted, but neither lever is statistically optimal across a dataset with a 100× or wider range in per-specimen read depth. A 5-read variant in a low-error context might be much more credible than a 50-read variant in a length-7 G homopolymer, but the tool had no way to express the difference.

This third concern motivated the variant significance framework (Paper 1 [1]) and its empirical extension (Paper 3 [3]). The mathematical content is in those papers; the pipeline machinery needed to act on the framework — a chain of upstream cleanup phases plus the CER test itself — is the subject of Section 7. The caveat below applies to the framework as a whole.

A careful caveat is in order before proceeding. Even with the CER framework, the pipeline cannot say which low-support variants are real — it can only say which ones are not artifacts of nanopore basecaller error. A variant that the framework rejects (`cer_factor` below threshold) is not necessarily wrong biology; it might be real biology that simply lacks the read support to distinguish from basecaller noise on this run. And a variant that passes the threshold is not necessarily real biology either; it might be a chimera, a contaminant, an upstream demultiplexing error, or any artifact whose generative process is something other than basecaller error. What the framework gives us is a clean separation along a single axis: "could this be basecaller noise?" — yes or no. Everything else remains the validator's call. In practice, validators answer those other questions through long-term, multi-run accrual of reference data: variants that recur across specimens and across sequencing runs accumulate evidence for biological reality that no single specimen's read pile can supply. The CER framework is a per-run filter; the validator's reference-database work is the cross-run integrator.

7 The 0.8.0 pipeline

The 0.8.0 pipeline runs in thirteen sequential phases. Six are new in 0.8.0: Phases 5–10, a coupled cleanup-and-routing cascade. Phase 9's MAD-based outlier removal shares its intention with 0.7's static-threshold outlier removal — pull a small tail of misfit reads out of an otherwise-clean cluster — but the underlying mechanism is different enough that it behaves like a new phase, and §8 shows it is the single largest quality contributor in the additive ladder. The remaining phases are largely

unchanged from 0.7 in mechanism but renumbered into the flat sequence. This section describes each phase and the design intent behind it; quantitative results live in Section 8.

The new phases form a connected cascade rather than a list of independent improvements. The CER framework (Phase 10) was the original target — a defensible statistical test for low-support variants. Implementing it required confidence that the variant positions being tested were real, which surfaced a chain of upstream issues. We needed the consensus that drives variant-position analysis to be trustworthy (Phase 5: noise filter — disband clusters whose consensus has positions with no majority). We needed reads to be assigned to the cluster they best fit, not merely the one MCL placed them in (Phase 6: read reassignment), and we needed reads earlier phases had discarded to get a second chance once the cluster landscape stabilized (Phase 7: discard recovery). When discard recovery brought new reads into existing clusters, it exposed variant positions the first phasing pass had missed (Phase 8: second phasing pass). Even after all of that, a small number of outlier reads in otherwise-clean clusters were inflating residual ambiguity, motivating a per-cluster cleanup (Phase 9: MAD-cleaned consensus generation) that runs immediately before CER so the statistical test sees the stabilized cluster state. The phases were designed together, and we will see in Section 8 that they only deliver their intended benefit *together*: turning some on without their upstream prerequisites can actively hurt.

Phase 1: Initial clustering

MCL on a K-NN similarity graph by default; greedy fallback if MCL is unavailable. Unchanged in mechanism since 0.6. Immediately after Phase 1, a *hard floor* of 3 reads is applied: clusters with fewer than 3 reads are dropped to discards, since reliable error correction requires at least three observations per position. This hard floor is not a configurable knob and applies regardless of the user-set `--min-size` (which can be set as low as 3 but no lower). Its purpose is structural rather than quality-driven: a 1- or 2-read "cluster" is not a consensus in any meaningful sense.

Phase 2: Pre-phasing merge

Combine clusters whose consensuses are equivalent under homopolymer normalization, applied via core's native HP-aware comparator with a configurable HP-length threshold (`--hp-normalization-length`, default 6). HP runs of length ≥ 6 are treated as HP context (their length variation is suppressed); shorter runs score normally, so a length-3-vs-length-4 difference is no longer absorbed silently. This catches cases where the initial clustering placed reads into separate clusters that resolve to the same biological sequence after consensus.

Phase 3: Variant phasing

For each cluster, build an MSA, find variant positions where qualifying bases reach the configured frequency threshold, and split reads by their allele combination. A cluster with no variant positions passes through unchanged. Three changes in 0.8.0 sharpen this step relative to 0.7: (a) a position can now qualify either across the full cluster *or* within a quality-biased sample (top-quality reads), which catches signal that low-quality reads were diluting out; (b) the SPOA alignment parameters are tuned to reduce the relative gap penalties at this step, which makes it easier for the alignment to distinguish a true substitution from a homopolymer-length artifact and exposes more variant positions cleanly; and (c) reads that do not match either the major or a phased minor allele cleanly are now sent to the discard pool for later recovery (Phase 7), where 0.7 had instead force-assigned them to the closest-matching sub-cluster (commit `372fa5b`, April 2026). The previous behavior

quietly contaminated otherwise-clean sub-clusters with reads that matched neither allele well and surfaced as ambiguity codes in the consensus.

Phase 4: Post-phasing merge

Run the pre-phasing merge logic again — same HP-length threshold, same HP-aware comparator — this time on the subclusters produced by phasing. Phasing can produce clusters that are HP-equivalent to clusters elsewhere in the specimen; this consolidates them.

Phase 5 (NEW): Noise filter

Run SPOA on each remaining small cluster. If the resulting MSA has columns where no base accounts for more than half of the reads, the cluster is disbanded — its consensus is not really a consensus. Concretely, a 4-read cluster with a 2-2 split at some position fails the test, as does a 3-read cluster with three different bases at a position; both patterns are common in genuinely noisy small clusters. Reads from disbanded clusters fall to the discard pool and may return via Phase 7.

Why this step: A cluster whose own consensus is unreliable cannot serve as a reference for downstream variant-position analysis (Phases 6, 7, 8) or pairwise statistical evaluation (Phase 10); better to disband it and let its reads find a more cohesive home in Phase 7. A second mechanism, less obvious but empirically substantial: noisy small clusters tend to generate spurious variant positions during MSA analysis, which then feed Phase 6's concordance scoring as if they were real signal. Removing those clusters prunes the false-positive variant positions before reassignment uses them.

Phase 6 (NEW): Read reassignment

Within each complete-linkage identity group, test whether each read agrees better with its current cluster's consensus or with another cluster's consensus in the same group. If concordance with a different cluster is meaningfully better, move the read. An edit-distance guard prevents reads from drifting across distinct sequences. Reassignment uses both edit distance and per-position concordance at variant sites — the same machinery that Phase 7 will reuse.

Why this step: MCL clustering is good but not perfect; reads can land in a sibling cluster they don't quite fit. The signal that lets us catch this in Phase 6 was not available in Phase 1: at initial clustering time we do not yet know where the meaningful variant positions are, so we have no better basis for read-to-cluster comparison than raw edit distance. By Phase 6, variant phasing has surfaced concrete variant positions, and we can compare each read's bases at those positions against each candidate cluster's consensus — a much more discriminating signal. Misassigned reads contribute their disagreements to the wrong consensus, inflating ambiguity; letting reads migrate to the cluster that best explains them at its variant positions tightens each cluster's internal consistency and reduces ambiguity counts.

Phase 7 (NEW): Discard recovery

Reads that were dropped earlier — failed phasing, MAD outlier removal, hard-floor filters, Phase 5 noise filtering — are tested against the surviving cluster consensuses. Reads that are concordant with a surviving consensus rejoin it. Auto-skipped if Phase 6 is disabled, since the two phases share machinery.

Why this step: Reads discarded earlier weren't necessarily bad reads — they often just didn't fit any cluster well at the time, because the cluster landscape was still in flux. After Phases 5

and 6 stabilize the surviving clusters, many discarded reads do fit a surviving consensus cleanly; admitting them increases support for real variants without changing what variants exist.

Phase 8 (NEW): Second phasing pass

If Phases 6 or 7 changed any cluster's membership, that cluster's MSA structure may have shifted enough to support a phasing decision that was not previously available. Re-run Phase 3 on those clusters. Gated by both `--enable-position-phasing` and `--enable-read-reassignment` — without either, nothing changed and there is no work to do.

Why this step: Discard recovery (Phase 7) routinely brings reads into existing clusters that surface variant positions the first phasing pass missed — either because the alternate allele was below the count threshold, or because the supporting reads weren't yet present in the cluster. Without a second pass, this newly-visible variation collapses to ambiguity codes in the consensus instead of resolving into separate variants. Phase 8 is primarily a quality tool (ambiguity reduction) and secondarily a discrimination tool (variant discovery): the same variation that fails to phase the first time through, when surfaced cleanly on the second pass, both reduces residual ambiguity in the larger cluster and produces a separate variant for the validator's review.

Phase 9 (NEW): MAD-cleaned consensus generation

Build an initial consensus for each surviving cluster, then remove reads whose identity to that consensus is more than $n\times$ MAD from the median — a small number of outlier reads can otherwise inflate the per-cluster disagreement reading on a cluster that is biologically clean. Rebuild the consensus on the surviving reads. *MAD* stands for median absolute deviation — for a cluster of read-vs-consensus identities, the MAD is the median of the absolute differences between each read's identity and the cluster's median identity. Reads whose identity sits more than several MADs away from the median are statistical outliers under any reasonable assumption about the underlying distribution, regardless of whether the cluster's reads are 99%-identity-clean or 92%-identity-noisy; this scale-free property is why the previous static `--outlier-identity` threshold is gone. Runs unconditionally; four CLI parameters control which reads are flagged (modified-Z threshold, gap rule, MAD floor, safety floor on absolute drop). §10 reports the 0.8.1 tuning of these parameters.

Why this step: Even after Phases 5–8 stabilize the cluster landscape, a small number of outlier reads in otherwise-clean clusters keep injecting per-position disagreement that downstream phases would otherwise see as residual heterogeneity. Running this cleanup before CER (Phase 10) and `err_factor` computation (Phase 12) means both downstream tests see a settled cluster state: CER's M and N read counts are post-MAD, and `err_factor` measures disagreement on the cleaned cluster. Without MAD, *any* choice of `--max-err-factor` is less discriminating: the threshold cannot cleanly separate completely problematic clusters from clusters that are good but happen to carry a few outlier reads, because both populations crowd into the same `err_factor` range. MAD pulls them apart, restoring the threshold's resolving power.

Phase 10 (NEW): CER validation

Within each identity group, the largest cluster is designated the *anchor*: it is presumed to represent the dominant biological signal in the group, and is exempt from significance testing. For every non-anchor cluster in the group, find its larger peers (the anchor and any other larger non-anchor) and compute the per-position context-aware critical error rate (CER) factor for each pair. Annotate the cluster with the minimum factor across peers — its weakest case. Anchors and clusters with

no comparable peer carry `cer_factor=None` and are exempt. Crucially, this phase only *annotates*. The pass/fail decision is applied later, by `speconsense-summarize`.

Why this step: The original target of the 0.8 redesign. The other new phases inflate the variant count (more variants surfaced from cleaner clusters), which would in turn inflate validator review burden if every variant passed through to the main output. CER provides the per-variant statistical test that lets the pipeline say "this small variant is almost certainly a basecaller-noise replica of a larger peer," routing demonstrably-bad variants to a separate track instead of dropping them.

Phase 11: Size filtering

Drop clusters below `--min-size` or `--min-cluster-ratio`. The Phase 1 hard floor at 3 reads still applies independently as a lower bound. The mechanism is unchanged from 0.7, but the defaults are not: 0.7.7 shipped `min-size=5` and `min-cluster-ratio=0.01`, and 0.8.0 relaxes both to `min-size=3` and `min-cluster-ratio=0`. The relaxation is intentional: the discrimination work that previously had to happen at this size-floor step has moved upstream to the noise filter (Phase 5) and downstream to CER routing (Phase 10) and the `err_factor` filter (Phase 12). Phase 11 in 0.8 is a safety net, not a primary filter.

Phase 12 (NEW): Output emission

Write the final FASTA, FASTQ, and MSA for each surviving cluster. Compute `err_factor` (the cluster's observed-vs-expected disagreement ratio under the basecaller error model) on the post-MAD MSA from Phase 9, and stamp it into the FASTA header for downstream routing in `speconsense-summarize`.

Why this step: Routing decisions (which records pass, which route to `.lq`) need a peer-independent quality metric, and `err_factor` provides it. Because the MSA was already built during the post-MAD consensus pass, no additional alignment work is needed here — emission is a thin wrapper over an existing artifact.

Phase 13: Discard reads written

When `--collect-discards` is set, write the discard pool to disk for inspection. Mechanism unchanged from 0.7.

7.1 `err_factor` and the `.ns`/`.lq` routing

Returning to the two cluster-level metrics introduced in Section 3.2, the 0.8 summarize tool routes records on each:

- In `speconsense-summarize`, records below `--min-cer-factor` (default 1.0) route to `__Summary__/variants/{name}.ns-RiC{ric}.fasta`.
- Records above `--max-err-factor` (default 1.5) route to `__Summary__/variants/{name}.lq-RiC{ric}.fasta`.
- `.lq` takes precedence when both fire. Neither filter drops records — they redirect them to a separate inspection track for later review.[1]

[1] In current practice, the lab sequencer running the primary data analysis (e.g., Stephen Russell at Mycota Lab, Harte Singer at FUNDIS) is the immediate consumer of the `.ns` and `.lq` tracks; downstream validators work on MycoMap, which at the time of writing exposes only the pass-track summary and not the `variants/` subdirectory. Surfacing the routed tracks through MycoMap is on the roadmap.

In the empirical results below, the two tracks behave as different inspection surfaces, not as parallel biological-recovery channels. `.ns` is dominantly basecaller-noise replicas of pass-track variants (§8.3: median `cer_factor` = 0.022). `.lq` is empirically a *within-specimen* variant-exploration track: in our 955-specimen run it adds zero specimens to pass-track recall (§8.7). Validators who look only at the pass track miss the within-specimen variant exploration but see no specimen-level recall loss; validators who look at `.lq` see additional variants for already-recovered specimens but no new organisms.

8 Empirical results

We report results from two empirical matrices, both run against the full ont98 ITS dataset (955 demultiplexed specimens; full methods in Section 9).[2] The first is an *additive ladder* on the v0.8.0 binary: starting from a stripped-down baseline (Phases 1, 2, 3, 4, 11, and 12, with Phase 9 MAD disabled), the new phases are turned on one at a time in dependency order. Each step measures the marginal contribution of one phase given everything that came before. The second is a *cross-version* comparison: v0.7.7 with its own defaults vs. v0.8.0 with its own defaults, the picture a validator sees when they upgrade.

8.1 Per-step contributions of the new phases

The additive ladder, on the validator-facing pass track. Baseline `B` = Phases 1, 2, 3, 4, 11, and 12-without-MAD enabled (the v0.7-equivalent core); each subsequent row turns on one new phase in dependency order, with `+MAD` (=v0.8) the terminal step. Each delta is the marginal contribution of one phase *given everything before it*; phases interact, so a different ordering would produce different intermediate values — §8.2 shows the alt-order trajectory and confirms that adding P7 before P5 actively hurts before P5 catches up.

Step	records		yield (reads)		ambig codes		ambig
	count	Δ	count	Δ	count	Δ	/rec
`B` (baseline)	3,767	—	315,128	—	1,108	—	0.294
`+P6`	3,552	−5.7%	317,451	+0.7%	1,043	−5.9%	0.294
`+P6+P5`	3,246	−8.6%	314,816	−0.8%	650	**−37.7%**	**0.200**
`+P6+P5+P7`	2,928	−9.8%	371,752	**+18.1%**	973	+49.7%	0.332
`+P6+P5+P7+P8`	3,355	+14.6%	366,877	−1.3%	799	**−17.9%**	0.238
`+MAD` (=v0.8)	3,689	+10.0%	366,625	−0.1%	927	+16.0%	0.251

The same ladder, viewed through the above-expected-disagreement lens (percentage of pass-track records and yield with `err_factor` > 1):

[2] The ablation matrices in §8.1–8.5 and the cross-tool tables in §8.6–8.7 were generated under the pre-reorder pipeline, where CER (then Phase 9) annotated clusters before MAD outlier removal (then Phase 11). On the post-reorder `v08-defaults` re-run reported in §8.4, the headline pass-record / ambiguity / yield numbers shift by 2–3% (and the .lq count by under 1%), without altering specimen-level recall. Full matrix re-derivation under the post-reorder ordering and the 0.8.1 MAD defaults is deferred to a future revision pass; the qualitative findings here hold under both.

Step	records err>1 (%)		yield err>1 (%)	
	share	Δ	share	Δ
`B` (baseline)	24.5	—	6.6	—
`+P6`	24.7	+0.8%	7.0	+6.1%
`+P6+P5`	21.4	**−13.4**%	6.5	−7.1%
`+P6+P5+P7`	27.1	+26.6%	9.7	**+49.2**%
`+P6+P5+P7+P8`	26.0	−4.1%	8.7	−10.3%
`+MAD` (=v0.8)	**16.6**	**−36.2**%	**2.6**	**−70.1**%

Reading the table: the ambig/rec column is the per-record cleanup rate, removing the confound that record counts shift between steps.

- **P5 (noise filter)** cuts pass-track ambiguity by −37.7% when added to a pipeline that already has P6: it is directly removing noise-blob clusters whose ambiguity codes were polluting the pass track. At the core source level, the same step cuts ambiguity by −84% (3,820 → 375 ambig codes per core record); most of P5's benefit happens upstream of the routing filters.

- **P7 (discard recovery)** is the yield engine: +18.1% pass-track yield, broadly distributed (676 of 828 specimens gain reads, only 81 lose). It does not significantly change variant count — the recovered reads consolidate into existing clusters — but its raise in ambig (+49.7%) is the cost: some recovered reads bring residual disagreement into otherwise-clean consensuses. The quality lens shows the same pattern: above-expected-disagreement yield jumps from 6.5% of pass to 9.7% as the recovered reads inflate the noise share.

- **P8 (second phasing pass)** resolves much of that residual disagreement: −17.9% ambig and +14.6% records. Variation that was collapsing to ambiguity codes in larger clusters is now phased into separate, smaller variants for validator review.

- **MAD outlier removal** is the dominant single quality contributor in the ladder. Its +10%-records bump is not new discovery: the per-cluster `err_factor` distribution tightens enough that records that previously routed to `.lq` now make it to pass. The above-expected-disagreement share drops from 26.0% to 16.6% of records (−9.4pp) and from 8.7% to 2.6% of yield (−6.1pp; a −70% relative reduction) in this single step. The new phases by themselves leave the noise share roughly flat against baseline; MAD does most of the quality lifting in the additive sequence.

A sanity check confirms the ladder is consistent with the original ablation matrix: `+P6+P5+P7+P8` matches a separate `v08-no-mad` ablation run (full 0.8 pipeline minus MAD) to within rounding noise (0 records / +21 reads / +4 ambig codes out of 3,355 records). The same set of phases enabled produces the same outputs regardless of how they were arrived at.

8.2 The new phases are a coupled cascade

The most striking result from the additive matrix is what happens when the phases are added in a different order. We re-ran the ladder with `P7` added before `P5` (the alternate sequence `B` → `+P6` → `+P6+P7` → `+P6+P5+P7` → ⋯) and compared the ambiguity-code trajectory to the linear ordering above:

ambig codes after →	+P6	next step	next step	= +P6+P5+P7
linear order (P5 before P7)	1,043	650 (+P5)	973 (+P7)	973
alt order (P7 before P5)	1,043	**1,136** (+P7)	973 (+P5)	973

Both paths land at the same end state (the same set of phases enabled produces the same output, deterministic), but the trajectory is sharply different. **Adding P7 to a pipeline without P5 actively increases ambiguity codes (+9% above +P6 alone).** Discard recovery, absent the noise-filter prepruning, brings noise-blob reads back into otherwise-clean clusters where they introduce new ambiguity. Adding P5 afterwards then cleans those up.

The mechanism behind the interaction is concrete: noisy small clusters generate spurious variant positions during MSA analysis, and those false-positive positions feed P6's concordance scoring as if they were real signal. Without P5 to disband the noisy clusters first, P6 reassigns reads using an inflated set of fake variant positions; P7 then re-admits discards on the same flawed basis. The combination of P5 + P6 + P7 only delivers its intended yield-and-ambig benefit when P5 has run first to prune the false positives.

The new phases are not independently optional: turning on P7 alone is worse than baseline, and the full benefit only appears when {P5, P6, P7, P8} are enabled together as a unit. This is the empirical fingerprint of the coupled-cascade design we described in Section 7.

8.3 Track separation in the routing dimension

Phase 10 (CER) and Phase 12 (`err_factor`) routing are summarize-only annotations, independent of the additive core ladder. On `v08-defaults` across the 955 ont98 specimens, of 13,300 core records:

track	records	median size (reads)
pass	3,689 (28%)	8
`.ns` (CER-rejected)	7,483 (56%)	4
`.lq` (err-rejected)	1,659 (12%)	3

Every core record is accounted for: 12,831 emit directly to pass/`.ns`/`.lq`, and the remaining 469 are conflated into 414 pass-track records via summarize's identity-group merging. No core records are dropped. The pass track ends up heavily-supported (mean = 99 reads, p90 = 387 reads); the rejection tracks contain qualitatively different populations of small clusters.

The `.ns` track is the strongest single empirical result of the study. Variants routed there have a `cer_factor` distribution with median **0.022** and p25 = 0.003: 65% of `.ns`-routed variants have `cer_factor` below 0.10 — orders of magnitude below the routing threshold of 1.0. The median `.ns` variant would need the basecaller's per-position HP error rate inflated 45-fold to plausibly arise as artifact. The CER framework is not just routing borderline cases; it is routing variants the math identifies as essentially-certain basecaller noise.

Phase 9's MAD step shapes the `err_factor` distribution and so shapes the `.lq` track. Median per-specimen `err_factor_p90` drops from 2.16 (without MAD) to 1.35 (with MAD enabled) — a −38% reduction in the tail. 76% of specimens benefit, 18% are unchanged, 6% see the per-specimen p90 inflate (a metric-shrinkage artifact: MAD removes outlier reads, the smaller surviving cluster has fewer reads diluting per-position disagreement, so the disagreement reading concentrates upward on a smaller denominator). The downstream effects are large: MAD reduces `.lq` routing by −51% (3,390 → 1,659 records), drops the share of pass records with above-expected disagreement

from 26.0% to 16.6%, and drops the above-expected-disagreement yield share from 8.7% to 2.6% (a −70% relative reduction). MAD is, empirically, the largest single quality contributor among the new phases.

8.4 Cumulative effect: v0.7.7 → v0.8.0

Stepping back: what does the cumulative v0.7-to-v0.8 upgrade look like to a validator running the same data through the new pipeline? Comparing v0.7.7 with default settings to v0.8.0 with default settings, on the validator-facing pass track:

metric	v0.7.7	v0.8.0	delta
pass-track variants	2,831	3,609	+27%
pass-track yield (reads)	348,399	366,168	+5%
pass-track ambiguity codes	1,615	902	−44%
records with `err_factor` > 1	830 (29%)	612 (17%)	−26%†
yield with `err_factor` > 1	15,174 (4%)	9,402 (3%)	−38%†
`.ns`-routed variants	—	7,574	new
`.lq`-routed variants	—	1,665	new
specimens with output	837	850	+13

† The `err_factor`-related v0.8.0 rows were derived from the pre-reorder extraction; on the post-reorder `v08-defaults` re-run, the pass-record count of `err_factor` > 1 shifts to 558 (a ∼0.1pp decrease in share). A consistent re-derivation for both columns under the post-reorder ordering and the 0.8.1 MAD defaults is deferred to a future revision pass.

The headline is one of *clearer separation*: 0.8 emits 27% more distinct candidate biological variants per specimen with 44% fewer ambiguity codes per consensus, and a small (5%) yield gain on top. The `.ns` and `.lq` tracks absorb 9,239 additional candidate variants — these were either silently averaged away in 0.7 (smearing the consensus) or filtered out by the per-cluster size threshold. Now they are visible to validators as separate, statistically-flagged tracks. Pass-track quality also improves: the count of records with above-expected disagreement (per-cluster `err_factor` > 1, i.e., observed read-to-consensus disagreement above what the basecaller error model alone predicts) drops from 830 to 612 records (−26%), and the corresponding yield drops from 15,174 to 9,402 reads (−38%). The +27% headline is gain in *trustworthy* records, not just any records.

The headline yield gain of +5% is heavy-tailed: median per-specimen delta is +3 reads, with 434 specimens gaining (p90 = +119 reads) and 376 losing (p10 = −42 reads). 0.8 is not uniformly better at recovering reads; it is better on average and substantially better on a subset.

The headline numbers above blend two distinct kinds of change: the new phases (mechanism), and a set of relaxed default parameters (permissivity). Section 8.5 decomposes the two; Section 8.6 sets both against an NGSpeciesID baseline run on the same dataset.

8.5 Permissivity vs. new-phase contribution

The 0.8.0 release ships with three loosened defaults relative to 0.7.7: `--min-size` (5 → 3), `--min-cluster-ratio` (0.01 → 0), and `--hp-normalization-length` (1, all-HP-normalized → 6, only HP runs ≥ 6 normalized). To isolate the new-phase effect from the permissivity effect, we run a third configuration, `v08-strict`: the v0.8.0 binary with v0.7-style strict thresholds (`--min-size 5`

`--min-cluster-ratio 0.01 --hp-normalization-length 1`). This represents "the new pipeline judged at 0.7's standard for what counts as a passing variant."

metric	v0.7.7	`v08-strict`	v0.8.0	new-phase Δ	permissivity Δ
pass records	2,831	2,215	3,689	**−21.8**%	**+66.5**%
pass yield (reads)	348,399	387,614	366,625	+11.3%	−5.4%
pass ambig codes	1,615	533	927	**−67.0**%	**+73.9**%
records w/ `err_factor` > 1	830	212	612	**−74.5**%	**+188.7**%
yield w/ `err_factor` > 1	15,174	5,275	9,402	**−65.2**%	**+78.2**%

Reading the columns left-to-right: v0.7.7 (baseline) → `v08-strict` (new phases at 0.7-style permissivity) → v0.8.0 (also relaxes the defaults). The penultimate column shows the new-phase contribution in isolation; the last column shows the permissivity contribution. The two effects pull in opposite directions on every metric.

The new phases by themselves tighten the variant set in the validator-preferred direction: fewer pass-track records (−22%), each larger and better-supported (+11% yield per variant); ambiguity codes slashed (−67%); records with above-expected disagreement cut by nearly three quarters (−74%); the noise-blob, force-assigned, and unphased-residual variants that 0.7 was emitting are largely caught and routed elsewhere.

The relaxed defaults push the variant count back up (+67% records), pulling in many smaller candidate variants that the strict thresholds would have rejected. Per-variant yield drops (−5%), ambiguity rises (+74%), and records with above-expected disagreement roughly triple (+189%) as a consequence; the smaller candidate variants are noisier than the larger anchors. Even after this permissivity rebound, however, the v0.8 default remains better than v0.7 on every quality axis: −43% ambig and −26% above-expected-disagreement records on a +30%-larger pass set.

The +30% pass-record headline of v0.7 → v0.8 is the sum of these two effects: +67% from looser thresholds, partially offset by −22% from the new phases doing their quality work.

Where the missing pass-yield reads go. The `v08-strict` → v0.8.0 pass-yield decrease (−5.4%, −20,989 reads) raised the question of whether reads were being lost. They are not. Total *core* yield in fact *increases* +3.4% (+13,620 reads) under v0.8 defaults; the apparent pass-track loss is reads being routed to the `.ns` and `.lq` inspection tracks rather than dropped:

core → track	`v08-strict`	v0.8.0	Δ reads
core total	396,385	410,005	+13,620
→ pass	387,614	366,625	−20,989
→ `.ns`	7,297	36,955	+29,658
→ `.lq`	1,474	6,425	+4,951

The bookkeeping balances exactly: $-20{,}989 + 29{,}658 + 4{,}951 = +13{,}620$. The mechanism is the chain we described in Section 8.2: looser HP normalization (`--hp-normalization-length 6` vs. 1) lets Phase 3 detect more variant positions at HP boundaries and split clusters that `v08-strict` kept whole. The smaller sub-clusters drop below 0.7-style thresholds but survive the v0.8 hard floor of 3 reads. They then route to `.ns` (CER rejects them as plausible basecaller-noise replicas of larger peers in the same identity group). The pass-track decrease is the new significance filter doing

exactly the work it was designed for, on a larger candidate population that the relaxed defaults made visible.

The takeaway for validators: the v0.7 → v0.8 "+30% variants" headline is permissivity-driven. The new phases by themselves produce *fewer*, *better-supported*, *less-ambiguous* pass-track variants than v0.7 did. The variant-count gain you see in v0.8-defaults is the new pipeline judging a larger candidate population and surfacing more of it.

HP-interface variants surfaced. Of the three permissivity changes, the move from `--hp-normalization-length 1` (v0.7 default, all HP runs normalized) to `--hp-normalization-length 6` (v0.8 default, only runs ≥ 6 normalized) is the one specifically motivated by the interface-homopolymer issue from Section 6. The validator-flagged class of variants is structurally a 1-base shift in an HP run's boundary — e.g., `ACCTA` vs. `ACTTA`, where the C/T interface has moved by one position, with each side a homopolymer "run" (in quotes because either side may be length 1, and the case generalizes to longer runs). Under v0.7's HP-1 normalization, both forms compared identically; the variant phaser never saw the interface position as variable, reads from both alleles ended up in the same cluster, and the resulting consensus either voted a majority HP-length (silently discarding the minor allele) or emitted an ambiguity code at the projected substitution position. Under v0.8's HP-6 normalization, HP runs of length 1–5 are no longer normalized, so reads with different HP-run lengths show real disagreement at the shift position; the phaser detects this and splits the cluster.

To measure how much such heterogeneity v0.7 was hiding, we built a phasing-implementation-independent detector that works directly on cluster MSAs (Methods). For each cluster, the detector identifies every consensus HP run of length ≥ 2; for each read it scans left and right of the consensus run, collapsing alignment gaps, to determine the read's actual HP-run length at that position; the cluster is flagged as having an unphased HP-interface if at least 10% of reads (and at least 3 reads, matching the speconsense phaser's variant-call thresholds) show a single-direction ±1 shift. Restricted to HP runs of length 2–5 (the population where v0.7 normalizes but v0.8 does not), and to pass-track core clusters only (the validator-facing output):

metric	v0.7-defaults	v0.8-defaults
pass-track core clusters analyzed	2,209	3,034
clusters with unphased HP-interface	598 (27.1%)	102 (**3.4%**)
pass-track specimens with ≥ 1 such cluster	353 (49.0%)	91 (**11.9%**)

Among pass-track clusters, the unphased-HP-interface rate drops from 27% in v0.7 to 3% in v0.8 — an 8-fold reduction — and the share of specimens whose pass-track output contains at least one such cluster falls from 49% to 12%, a 4-fold reduction. Most of v0.7's heterogeneous clusters now phase into separate v0.8 sub-clusters with their own consensuses, exposing the HP-interface variants v0.7 was hiding. Two cross-checks support the detector: a structural enumeration of within-specimen pass-track variant pairs (Methods) finds 7 pairs across 5 specimens in v0.8 that differ only by a length-conserving canonical HP-interface shift (the strict `ACCTA` vs. `ACTTA` case); v0.7 emits zero such pairs. Four of those five specimens are independently flagged by the MSA-based detector applied to v0.7's pass-track clusters; the fifth has a v0.7 cluster showing HP-shift heterogeneity at sub-threshold magnitude (small sample size, fewer than 3 reads in the shift direction), consistent with the phaser's read-count minimum being the binding constraint.

The 91 v0.8 specimens with residual unphased HP-interface clusters are mostly cases where the cluster's MSA is too small for the phaser's read-count minimum, plus cases where the HP-shift

co-occurs with other variation that breaks the single-direction signal. The first is fundamental; the second is plausibly addressable in a future revision.

8.6 Cross-tool baseline: NGSpeciesID

NGSpeciesID was developed for a different workload than speconsense addresses. Sahlin and colleagues [6] optimized for biodiversity-scale processing of single-amplicon specimens with minimal preprocessing; isONclust [7], the clustering algorithm at NGSpeciesID's core, uses minimizer hashing to skip pairwise alignment for most reads, trading some clustering precision for the throughput needed to process thousands of samples on a laptop. The design assumption is one consensus per specimen — the `--abundance_ratio` default of 10% deliberately suppresses minor variants — and the authors explicitly note mixed-sample handling as outside the design target.

The Russell protocol's invocation, refined over Mycota Lab's production use, already opts out of much of that tradeoff: `--mapped_threshold 1.0` disables isONclust's minimizer-based shortcut entirely (the threshold becomes unsatisfiable), forcing parasail alignment for every clustering decision; `--symmetric_map_align_thresholds` requires the alignment ratio to clear the threshold from both directions; and `--abundance_ratio 0.2` relaxes the variant-suppression default. We ran NGSpeciesID 0.3.0 with medaka 2.0.0 polishing on the same 955-specimen ont98 dataset, with the full invocation `--ont --consensus --symmetric_map_align_thresholds --aligned_threshold 0.75 --mapped_threshold 1.0 --medaka --abundance_ratio 0.2 --sample_size 500 --top_reads`. What we compare against speconsense is therefore a precision-tuned NGSpeciesID pipeline — greedy clustering by parasail alignment, SPOA consensus, medaka polishing, primer trim — not isONclust-as-shipped.

NGSpeciesID numbers in the table below are unfiltered (every emitted record, including RiC<3); the Mycota Lab production protocol applies a RiC≥3 post-filter to NGSpeciesID output before downstream use, which §8.7 analyzes in parallel.

metric	v0.7.7	v0.8.0	NGSpeciesID
specimens with output	837	850	**926**
pass records	2,831	3,689	1,180
pass yield (reads)	348,399	366,625	254,395
pass ambig codes	1,615	927	**0**
records / specimen (mean)	3.38	4.34	1.27
records, `err_factor` > 1	830 (29%)	612 (17%)	**431 (37%)**
yield, `err_factor` > 1	15,174 (4%)	9,402 (3%)	**78,019 (31%)**

NGSpeciesID is much more conservative on per-specimen variant discovery (median 1 variant/specimen, mean 1.27) than either speconsense version. This is by design: NGSpeciesID is built around the "one consensus per specimen" use case, with its `--abundance_ratio` filter cutting clusters below 20% of the dominant cluster's read count. The variants speconsense surfaces beyond NGSpeciesID's count are the intragenomic copies, paralogs, and minor allelic variation that motivated speconsense in the first place.

NGSpeciesID's wider specimen coverage — 89/76 more specimens with output than speconsense 0.7/0.8 — is partly a like-for-like artifact. The counts above are unfiltered, and the Mycota Lab production protocol post-filters NGSpeciesID's output to RiC≥3 because single- and two-read clusters are too uncertain to deposit (see §8.7). After that filter, NGSpeciesID and v0.8.0 cover essentially the same specimens (850 each, of 955). The residual gap traces to speconsense's clustering splitting low-depth read pools into sub-3-read clusters that fall below the emit threshold,

where NGSpeciesID's greedy single-cluster-per-specimen design produces a consensus regardless. The trade-off is real — speconsense provides finer-grained variant resolution at the cost of some specimens with very low input depth getting no output — but it is much narrower than the unfiltered numbers suggest, and what remains is concentrated in exactly the single-read-anchored evidence the protocol filter distrusts. Cross-referencing the 82 specimens that NGSpeciesID's raw output covers but v0.8.0 does not against §8.7's taxonomic agreement: 87% of the 281 records those specimens contribute are noise, only 10% hit a primary target, and 56 of the 82 specimens are truth-unknown to begin with. Most of NGSpeciesID's wider raw coverage is noise emit from low-quality specimens, and the protocol filter correctly rejects almost all of it.

The wider coverage comes with lower per-record quality, however. We recomputed `err_factor` for each NGSpeciesID record by sampling 100 supporting reads, building an MSA via SPOA, and applying speconsense's HP-aware comparator (the same code path the 0.8 pipeline uses internally; see Methods). 37% of NGSpeciesID's records show `err_factor` > 1 and 31% of its yield sits in those records — roughly **12× v0.8.0's** 3% **yield share** above the basecaller error model's expectation. `err_factor` is a model-relative metric computed under speconsense's HP-aware comparator and Dorado-v5.0 `q_ctx` tables; it measures internal consistency under speconsense's error model rather than absolute consensus quality. The records-vs.-yield ratio is informative on its own: in v0.7/v0.8, above-expected-disagreement records are concentrated in small clusters (records-share $\gg$ yield-share), but in NGSpeciesID the above-expected records are also the large clusters (records-share $\approx$ yield-share). NGSpeciesID has no equivalent of CER routing or `err_factor` filtering; records that speconsense would route to `.ns` or `.lq` land directly in the pass output. A validator using NGSpeciesID would need to apply downstream quality filtering to recover comparable trustworthiness.

NGSpeciesID emits zero ambiguity codes by design: it has no mechanism for surfacing residual within-cluster heterogeneity. A public reference database may prefer the cleaner consensus; a validator looking for unresolved heterogeneity loses information. Speconsense's ambiguity-code emission, paired with the noise filter and the significance framework, is one of the deliberate departures from the NGSpeciesID model.

The differences in the table reflect different points in a design space, not a defect on either side. NGSpeciesID was a good fit for getting Mycota Lab's nanopore protocol off the ground, and remains a sensible default for workflows whose primary target is one consensus per specimen at biodiversity scale. The validator community's evolving requirements — finer variant resolution, defensible filtering of low-support variants, ambiguity-code emission for residual heterogeneity, and transparent rejection tracks — pulled the workflow toward a different optimization target.

8.7 Taxonomic agreement

The §8.6 comparison is at the level of records and specimens — counts and quality measures. The biology question is whether those records correspond to real organisms. We added a taxonomic-agreement step that classifies each emitted variant against a per-specimen truth set: validator-confirmed organism IDs from the iNaturalist observations behind 935 of the 955 ont98 specimens. The truth set was built from a five-stage pipeline (vsearch + adjusted-identity search against MycoMap, UNITE escalation for unknowns, an internal reference of validator-confirmed (BISON, ITS) deposits, plus iNat metadata and validator-comment review) and then manually walked, row by row. After review, 857 specimens have a confirmed primary, 78 are truth-unknown (specimen failures, contaminations, or validator-flagged mixups), and 54 carry a recovered mycoparasite or co-occurring organism alongside the primary. Methods §9 describes the pipeline. One framing note: the ont98 reference is built from validator-confirmed deposits in the same dataset, so a specimen

matching its own ont98 deposit is a self-consistency check (the pipeline's consensus reproduces the validated lab call from the same reads) rather than a generalization test. We allow it because it is the only way to score temp codes that haven't yet reached the public reference — most temp codes do appear in MycoMap, but a long tail of recent or rarely-deposited ones do not. The MycoMap self-match filter still applies for the specimens whose validator-confirmed call exists in the public reference.

Each emitted variant lands in one of four buckets that sum to 100% per track. **primary_target**: the variant canonically matches the truth primary, or shares its genus (the same-genus extension absorbs cases where the reference DB has only a genus-rank deposit, e.g. "Xylodon sp." covering "Xylodon asper"). **myco**: a parasite, host, or co-occurring organism on this specimen's substrate (*Trichaptum biforme* on a *Phaeocalicium polyporaeum* collection; *Ampelomyces sp.* on *Golovinomyces*; the *Syzygites megalocarpus* mycoparasite of an *Agaricus* host that itself failed to amplify). **contaminant**: known yeast or upstream-process artifact. We treat cross-bleed contamination (e.g. a high-RiC *Hemileccinum hortonii* bleeding across multiple unrelated specimens) as a contaminant rather than noise: it is an artifact in the input data that any tool could in principle find, not a tool-produced one. **noise**: everything else — low-adj non-matches, family-level fallbacks, unexplained sequences — potentially tool-produced.

Recall is defined as (specimens with at least one primary-target hit) / 955, computed against the full 955-specimen input cohort.[3] Three failure modes contribute to non-recovery, and all count against recall: 20 of the 955 specimens produced no record from any tool (low-yield wells with reads that fail to cluster cohesively — median 19 input reads each, mean read length ∼1,300 bp vs. ∼700 bp for the cohort, suggesting concatemers or off-target amplification); 78 of the remaining 935 truth-set specimens are truth-unknown (specimen failures, contaminations, validator-flagged mixups); and a tool may emit records that simply do not match the truth primary. Pooling these on the same denominator means coverage gaps surface as quality differences rather than vanishing from the comparison.

NGSpeciesID's protocol filter splits the comparison. The Mycota Lab production protocol post-filters NGSpeciesID's output to RiC≥3, treating single- and two-read clusters as too uncertain for downstream use. We apply that filter and treat the rejected ric<3 emits as an `lq` track for parallel comparison with v0.8's `.lq`. The case for the filter is two-fold. A single read cannot be corrected for nanopore error, so even when the species call is right, the sequence cannot be deposited in a reference database — the very pipeline that produces our internal ont98 reference enforces this constraint. And single-read cross-specimen carry-over is common; we have at least one documented case (a *Hygrocybe*) where a validator initially selected a ric=1 sequence to identify the specimen, which in retrospect was a different *Hygrocybe* from another well's read pool, while a second ric=1 sequence in the same specimen actually matched the morphology. Our 20/20-hindsight truth set absorbs that particular swap, but the protocol cannot afford to.

In the table below, the `ngsi-defaults pass` and `lq` rows are the RiC≥3 / RiC<3 partition of NGSpeciesID's output (873 + 307 = 1,180 unfiltered records, matching §8.6). The `ngsi-defaults lq` row's recall (2.3%) and noise share (88.3%) look unhelpful in isolation; the paragraph "The `.lq`-track question" below unpacks the 22 specimens that the RiC≥3 filter would lose were it not for those records.

[3]The sequencing run loaded 960 specimens; 5 had no reads after demultiplexing.

run / track	cover	recall	records	%prim	%myco	%cont	%noise	lq_resc
ngsi-defaults pass	89.1%	83.7%	873	94.6%	0.7%	0.9%	3.8%	22
ngsi-defaults lq	10.2%	2.3%	307	8.1%	1.9%	1.6%	88.3%	—
v07-defaults pass	87.6%	85.1%	2,831	85.5%	1.0%	2.5%	11.1%	0
v08-strict pass	87.7%	85.3%	2,215	85.5%	1.1%	2.4%	10.9%	1
v08-defaults pass	**89.0%**	**86.3%**	3,689	82.1%	1.1%	2.4%	14.4%	0
v08-defaults `.ns`	70.6%	69.0%	7,483	97.6%	0.4%	0.9%	**1.0%**	—
v08-defaults `.lq`	52.5%	45.3%	1,659	89.3%	0.5%	2.0%	8.1%	—

The recall ranking matches the coverage observations from §8.5/§8.6 but at the cohort level. v0.8.0-defaults wins (86.3%) by combining the highest emit coverage (89.0%) with comparable per-emitting recovery to the strict variants. v0.7-defaults and v0.8-strict are essentially tied at ∼85% — the +1.0-pt improvement v0.8-defaults shows over its strict variant comes from the additional specimens its permissive emit recovers, not from any in-emit recovery improvement. Permissivity, not the new phases, is the recall lever.

The %noise column is where the variant-detection vs. consensus design difference shows up. NGSpeciesID is built around one record per specimen, and 94.6% of its records hit the primary target with only 3.8% noise. v0.8-defaults emits 4.3 records per specimen on average and 14.4% of its pass records sit in the noise bucket. The naive read of those numbers ("v0.8 is nearly four times noisier") misses what the records actually are, however: the noise concentrates dramatically by identity group.

Noise concentrates in groups 2+. For the v0.7/v0.8 tools, an identity group is the cluster a record came from — group 1 is the rank-1-by-RiC cluster per specimen, where the primary target is expected to live; groups 2+ are the additional clusters surfaced by the variant-discovery pipeline.

run / track	group	records	%prim	%noise
v07-defaults pass	1	2,438	96.9%	1.8%
	2+	393	14.2%	68.4%
v08-strict pass	1	2,079	90.0%	7.5%
	2+	136	16.9%	63.2%
v08-defaults pass	1	3,040	**97.2%**	**1.8%**
	2+	649	11.4%	73.5%
v08-defaults `.ns`	1	7,361	**98.9%**	**0.2%**
	2+	122	23.8%	52.5%
v08-defaults `.lq`	1	1,518	96.6%	2.8%
	2+	141	11.3%	65.2%

A validator inspecting only the rank-1 cluster sees noise drop from 14% to 2% in v0.8.0's pass track, and from 1% to 0.2% in the `.ns` track. The group-2+ records do carry real biology — 11–17% are primary-target hits and an additional 4–7% are myco-associates or known contaminants — but the majority are cross-bleed or unexplained. NGSpeciesID emits at most one consensus per specimen-cluster (no phasing within), so this analysis is specific to the variant-discovery tools. The validator-facing implication is that group 1 alone covers nearly all the recall while contributing a small fraction of the noise; groups 2+ are worth scanning for co-occurring biology but are noisier per record.

The `.lq`-track question. Both speconsense `.lq` and NGSpeciesID-protocol-filtered ric<3 represent records the tool emitted but the protocol/routing flagged as low quality. Do those records ever recover specimens the pass track misses? For NGSpeciesID, 22 specimens are recovered *only* by ric<3 emits — specimens whose only sequence evidence is a single-read or two-read cluster. The ric≥3 filter rejects these entirely; without the `lq` treatment they would be invisible recall failures. Of the 7.9-percentage-point coverage drop between NGSpeciesID's raw (97.0%) and protocol-filtered (89.1%) outputs, 22 specimens correspond to validator-confirmed primaries the protocol filter loses. The other side of that ledger: the 75 specimens lost to the filter overall contribute 264 records, of which 87.5% are noise — the filter rejects 22 valid recoveries while also rejecting ∼240 noise records. The two-fold filter justification still holds, and the trade favors the filter, but the recall cost is real and visible.

For v0.8.0, 0 specimens are rescued by `.lq`: every specimen the `.lq` track recovers is already covered by pass. `.lq` is purely a within-specimen variant-exploration track — it surfaces additional clusters for already-recovered specimens, not new specimen coverage. The asymmetry highlights how each tool handles low-quality clusters: NGSpeciesID's ric<3 emits include a tail of legitimate single-read specimens that protocol post-filtering loses, while v0.8.0's MCL clustering and routing capture the same specimens through normal pass channels.

Mycoparasites and substrate co-occurrences. 54 specimens have a recovered organism alongside the primary that the truth set classifies as a mycoparasite, host, or substrate co-occurrence. The cleanest examples are the obligate parasitic relationships: *Phaeocalicium polyporaeum* growing on *Trichaptum biforme* (the validator's iNat note recorded only the *Trichaptum* sequence, but v0.8.0 cleanly phased both the Phaeocalicium target and the Trichaptum host); *Ampelomyces sp.* on a *Golovinomyces* powdery mildew (the validator separately created a duplicate observation for the mycoparasite); and *Syzygites megalocarpus* on an *Agaricus* that itself failed to amplify but where the parasite came through cleanly. Recall on the mycoparasite sub-cohort tracks the per-record %myco column: NGSpeciesID's abundance-ratio default suppresses minor variants and rarely surfaces co-occurring organisms (0.7% myco), while v0.8.0's variant phasing preserves them at the same rate as v0.7 (∼1.1%) but on a larger record base.

9 Methods

Dataset. ont98: 955 demultiplexed per-specimen FASTQ files from a single Mycota Lab production run, Russell ITS amplicon protocol [5], Dorado v5.0 SUP basecalled. Same primer set (ITS1F / ITS4) across all specimens. Per-specimen read counts range from a few hundred to a few thousand.

Cross-version and MAD-ablation runs. Two comparisons drawn from fresh core runs underlie the body's quantitative claims outside the additive ladder:

Comparison	Reference	Run
Cross-version (Sections 8.4–8.7)	v0.7.7 defaults	v0.8.0 defaults
Phase 9 MAD (Sections 8.1, 8.3)	v0.8 defaults	`--disable-mad-outlier-removal` (`v08-no-mad`)

The `--disable-mad-outlier-removal` flag was added to the speconsense codebase specifically to support this study; it ships as part of 0.8.0. Per-phase subtractive ablations for the other new phases were also run during pipeline development but are superseded by the additive ladder below for the body's analyses.

Permissivity-isolation run. One additional fresh core run, `v08-strict`: the v0.8.0 binary with v0.7-style restrictive defaults (`--min-size 5 --min-cluster-ratio 0.01 --hp-normalization-length 1`) and otherwise default summarize. Used in Section 8.5 to decompose the v0.7 → v0.8 delta into new-phase and permissivity components.

Cross-tool baseline. NGSpeciesID 0.3.0 with medaka 2.0.0 polishing, run via the local checkout[4] at `~/mm/code/NGSpeciesID` with the Russell-protocol invocation: `--t 1 --ont --consensus --symmetric_map_align_thresholds --aligned_threshold 0.75 --mapped_threshold 1.0 --medaka --abundance_ratio 0.2 --sample_size 500 --top_reads --primer_file $PRIMERS`. Output collected into mycomap-format summary via the same `summarize.py` that Mycota Lab uses in production. Used in Section 8.6.

Additive-ladder matrix. Six fresh core runs on the v0.8.0 binary, starting from a stripped-down baseline and adding one new phase at a time:

Run ID	Disable flags (everything else default)

`v08-add-base`	`--disable-noise-filter`
	`--disable-read-reassignment`
	`--disable-mad-outlier-removal`

`v08-add-p6`	`--disable-noise-filter`
	`--disable-discard-recovery`
	`--disable-second-phasing`
	`--disable-mad-outlier-removal`

`v08-add-p6-p5`	`--disable-discard-recovery`
	`--disable-second-phasing`
	`--disable-mad-outlier-removal`

`v08-add-p6-p7`	`--disable-noise-filter`
	`--disable-second-phasing`
	`--disable-mad-outlier-removal` (alt-order)

`v08-add-p6-p5-p7`	`--disable-second-phasing` `--disable-mad-outlier-removal`
`v08-add-p6-p5-p7-p8`	`--disable-mad-outlier-removal`

The terminal step (full v0.8.0 with all phases enabled, including MAD and CER/err_factor routing) is the existing `v08-defaults` run from the subtractive matrix, reused. Each additive run is also re-summarized in *permissive* mode (`--min-cer-factor=0 --max-err-factor=0`) so that core conservation can be verified unambiguously. Phases 10 and 12's CER/err-factor routing are summarize-only and apply uniformly across the additive ladder via the default summarize step.

Cross-version `err_factor`. For v0.7 runs (where `err_factor` is not present in FASTA header output), values were computed offline from the per-cluster `cluster_debug/*-msa.fasta` files using the canonical 0.8 implementation. Cross-validation against the FASTA-header values for v0.8 runs showed agreement to within floating-point rounding (max abs diff 0.0005 across all 13,300 v0.8 core records).

NGSpeciesID `err_factor`. NGSpeciesID does not compute `err_factor`, so values were derived externally for the cross-tool comparison. For each NGSpeciesID record we sampled up to 100 supporting reads from `clusters/<specimen>/reads_to_consensus_<id>.fastq` (matching speconsense's production sample size), built an MSA via SPOA with the

[4] `https://github.com/joshuaowalker/NGSpeciesID` — the brew/PyPI 0.3.0 release does not include the `--symmetric_map_align_thresholds` or `--top_reads` flags used in the Russell protocol.

same options speconsense uses internally (`-r 2 -l 1 -m 1 -n -1 -g -1 -e -1`), and applied `compute_cluster_err_factor` from `speconsense.msa` — the canonical 0.8 implementation, with the same `dorado-v5.0` error-model table. The MSA reference is SPOA's own consensus from the sampled reads, so the value reported is the per-cluster read-to-consensus disagreement ratio that `err_factor` measures throughout this paper.

HP-interface analysis. Two complementary detectors targeted the interface-homopolymer effect. (1) *Structural pair enumeration.* For each run we enumerated every within-specimen pair of pass-track records and classified them structurally: each sequence was decomposed into a list of (base, run-length) tuples, and pairs were classified by relating the two decompositions. A pair was tagged `hp_interface_1_5` if the two decompositions had identical shape (same number of runs, same base at each position), all length-differing runs were ≤ 5, and the differences came in adjacent $+N/-N$ pairs (the canonical length-conserving boundary shift `ACCTA` vs. `ACTTA`); `hp_extension_1_5` if shape matched and all differing runs were ≤ 5 but the differences were not strictly adjacent $\pm N$ pairs (single-run length changes, non-adjacent shifts). Implementation in `validation/scripts/hp_interface_pairs.py`; per-pair output in `validation/analysis/hp-interface-pairs.csv`. (2) *MSA-level unphased-interface detection.* The structural enumeration is conservative: it only catches pairs whose differences are exclusively HP-shift, missing real HP-interface variants that co-occur with other variation. The MSA-level detector, applied to each cluster's `cluster_debug/<id>-msa.fasta`, addresses this. For each consensus HP run of length ≥ 2, the detector computes each read's actual HP-run length at that position by anchoring at the consensus run's MSA columns, walking outward in the read while collapsing alignment gaps, and stopping at the first non-gap base that is not the run's character. The cluster is flagged as having an unphased HP-interface at that run if at least 10% of reads, and at least 3 reads, show a single-direction ± 1 shift relative to the consensus's run length — matching the speconsense phaser's variant-call thresholds. Reads with multi-base shifts (± 2 or more) and reads with substitutions inside the run are excluded from the shift count to avoid conflating HP-interface signal with other variation. Implementation in `validation/scripts/hp_interface_msa.py`; per-flagged-run output in `validation/analysis/hp-interface-msa-v07.csv` and `-v08.csv`.

Taxonomic agreement. The §8.7 truth set was constructed in five stages. (1) A union FASTA was built by deduplicating every consensus emitted across all run/track combinations by sequence SHA-1, yielding ∼30k unique sequences against the four-tool × all-track product. (2) Each unique sequence was searched against three references in priority order: the MycoMap fungal-ITS reference (103,465 records), UNITE 19.02.2025 (2.07M records, used for unknowns at MycoMap-best adj<0.95), and an internal ont98 reference of 576 records built from iNaturalist observations where both the validator-confirmed BISON ("Barcode Inferred Species or Name") and the DNA-Barcode-ITS field were populated, plus two manual additions for specimens where the validator confirmed a species in comments without setting BISON. The search uses vsearch `usearch_global` as a fast filter followed by a per-hit pairwise rescoring with the HP- and IUPAC-aware `adjusted-identity` comparator (the same comparator the 0.8 pipeline uses internally). The MycoMap self-match filter is keyed by iNat ID embedded in MycoMap accessions: a specimen's own MycoMap deposit is excluded from its candidate set so that recall against an independent public reference is not measured trivially. The ont98 internal reference is deliberately exempt from this filter — its records are validator-confirmed (BISON, ITS) pairs from the same dataset, and self-matching is the mechanism by which we score validator-confirmed temp codes that have not yet reached MycoMap — most temp codes do appear in the public reference, but a long tail of recent or rarely-deposited ones do not. Matching a specimen against its own ont98 deposit is therefore a self-consistency check (does the pipeline's consensus reproduce the validated lab call from the same reads?) rather than a generalization test against an independent reference. We treat it that way in the results. (3) iNaturalist

metadata was pulled for each specimen via the v1 API — community taxon, OFV (observation field) values, comments, and identifications — with comments and identifications tagged by validator role from a manually-curated rank list of validator and expert iNat usernames. (4) A draft truth file was synthesized by combining BISON, PSN, the iNat species/genus, the search hits, and validator commentary, with thirteen boolean flags surfacing rows needing human review (genus-only matches, morphology-only validation, high-RiC non-primary candidates, and so on). (5) The 935-row draft was manually walked by the author. Review was *not* blinded to tool output — the draft surfaces per-tool routing decisions alongside iNat metadata and sequence-search hits, and the author had context on which records came from which run — so the truth set should be read as an author-curated reference standard, not a blinded ground truth. Adjudication priority was: validator-confirmed BISON or DNA-Barcode-ITS values from iNat; iNat community-taxon species rank with sequence agreement; validator commentary on the specimen; and biological plausibility against MycoMap, UNITE, and ont98 search hits. ∼574 trivial cases (BISON-validated or iNat species match) were spot-checked and bulk-accepted; the remaining ∼361 decision rows were resolved one by one. The final truth set assigns each specimen a confirmed primary taxon (or truth-unknown), a list of mycoparasites/co-occurring organisms, and a list of contaminants, plus a confidence tag distinguishing high-confidence calls (validator-confirmed via BISON or DNA-Barcode-ITS) from low-confidence calls (e.g. NGSpeciesID-only ric<3 emits where the truth assignment depends on the very record being scored). All truth-construction scripts and the final truth.csv are committed under `validation/scripts/` and `validation/analysis/taxonomy/`.

Reproducibility. All metric extraction, aggregation, and pairwise comparison scripts, along with the per-run aggregate CSVs and per-hypothesis comparison tables, are committed under `validation/` in the paper repository.

10 Looking ahead

The 0.8.0 architecture is built around four ideas: annotate everything, decide downstream; separate noise filtering from significance testing; phase variants aggressively to reduce ambiguity codes; and design the new phases as a coupled cascade rather than a list of independent options. The first makes the pipeline replayable — a validator who disagrees with a default threshold can change it without re-running the (expensive) clustering and consensus steps. The second makes the validator's mental model match the pipeline: `err_factor` for "is this cluster real?" and `cer_factor` for "is this variant distinguishable from its peers?" are the two questions a validator naturally asks, and the pipeline now produces a number for each. The third inverts the older default: where 0.7 reached for an ambiguity code whenever phasing was inconclusive, 0.8 makes a sustained effort to phase the variation cleanly through Phase 3, the noise-filter-mediated cleanup of Phases 5–7, and the second-pass phasing of Phase 8 — and only emits an ambiguity code when none of those mechanisms can resolve the disagreement.

The empirical results above support all four concretely. The CER framework's `.ns` track contains variants whose `cer_factor` median is 0.022 — not borderline calls but variants the math identifies with high confidence as basecaller-noise replicas. Validators previously dismissed these by hand; the pipeline now does the routing for them. Phase 9's MAD outlier removal compresses the `err_factor` distribution enough to make `--max-err-factor=1.5` a defensible default (§8.3). Discard recovery (Phase 7), once Phase 5 has filtered the discard pool, contributes a +18% pass-track yield gain. Every core record is accounted for in pass, `.ns`, or `.lq` (with summarize identity-group merging conflating some pass-track variants), making every routing decision auditable.

The fourth idea was less visible in the design but is visible in the data: the new phases were

designed as a coupled cascade, and the additive matrix shows that they are. Adding any single phase in isolation can move results in unexpected directions — notably, adding discard recovery (Phase 7) without first adding the noise filter (Phase 5) actively increases pass-track ambiguity, because unvetted reads from noise-blob clusters get pulled back into otherwise-clean ones. The intended benefit only appears when the P5, P6, P7, P8 group is enabled together. The ablation flags shipped with 0.8.0 are intended for studies of this kind, not as production knobs — a user who wants to keep one of these phases off in production is, on the evidence, asking for a different tool.

What remains for future work breaks into two horizons: a near-term 0.8.1 fast-follow that ships parameter-tuning refinements identified during the 0.8.0 validation, and a 0.8.2 revision focused on the defaults that were left as deliberate first cuts pending production-cohort evidence.

The 0.8.1 fast-follow ships a tuned MAD outlier-removal default. A sweep of the four MAD CLI parameters across the full 955-specimen cohort found that the four knobs split into two functional families with opposing effects on `err_factor`'s discriminative power. The shape-aware rules (`--mad-z-threshold`, `--mad-gap-factor`) flag statistical outliers within a cluster regardless of magnitude — selectively present in primary clusters whose heterogeneity comes from sequencing noise around a coherent consensus, and largely absent from noise clusters whose heterogeneity is distributed across all reads. Tightening these rules tightens primary-cluster `err_factor` distributions without disturbing noise clusters, increasing the separation between the two populations and making any choice of `--max-err-factor` a more accurate classifier. The magnitude-aware safety floor `--mad-min-drop-from-median` does the opposite: it compresses `err_factor` uniformly across primary and noise clusters, decreasing the routing classifier's accuracy even though individual cluster consensuses look cleaner. The fourth knob (`--mad-min-mad`) had no detectable effect at any tested setting. On the basis of the sweep, 0.8.1 tightens `--mad-z-threshold` from 3.0 to 1.5: the AUC of `err_factor` as a noise-vs-primary classifier rises from 0.63 to 0.66, the F1 of the `--max-err-factor=1.5` routing decision rises by 30%, and per-specimen primary recall moves by one specimen (within slack).

The 0.8.2 revision targets the routing and size-based defaults left as first cuts in 0.8.0. `--min-cer-factor=1.0` is theoretically grounded as the basecaller-noise line and is unlikely to move. `--max-err-factor=1.5` and the size-based thresholds (`--min-size`, `--min-cluster-ratio`) are safe but unoptimized; the `v08-strict`/`v08-defaults` contrast in §8.5 highlights the size-threshold question in particular, where the right operating point sits somewhere between the two. A joint sweep of `--max-err-factor` under the 0.8.1 MAD configuration found that tightening to 1.0 or 1.2 costs 6–12 primary recoveries and is not currently defensible; size thresholds remain on the watch list pending production-cohort evidence. We expect 0.8.2 to settle those defaults based on key-partner review, validator feedback, and user-acceptance testing on additional production cohorts.

The longer horizon includes refreshing the error-model tables as basecallers evolve, validating the CER framework against orthogonal datasets, and exploring whether the framework extends gracefully to non-amplicon use cases. The most open challenge is downstream: choosing how to surface the multi-track output (pass / `.ns` / `.lq`) to validators and to public reference databases in a way that preserves the information content the pipeline has worked to surface, without overwhelming downstream consumers who are used to one consensus per specimen. Work the pipeline has done to expose biological variation cleanly is only useful if the consumers of its output can act on it.

Acknowledgements

The CER framework grew out of a sustained conversation between Stephen Russell (Mycota Lab), Harte Singer (Fungal Diversity Survey), and the author, all three concerned about where to draw the line between a real low-support variant and basecaller noise; specific cluster-level questions Stephen flagged about specific specimens, and the SNP-calling-at-scale problem Harte raised in August 2025, motivated the work directly. Two sequence validators in particular drove the recurring questions of Section 6 into the empirical work behind 0.8.0: Elora Adamson, Local Coordinator for the MycoMap BC Network project (British Columbia), surfaced both the interface-homopolymer issue that drove the context-aware error model and the ambiguity-inflation issue that drove the noise filter, with a running set of cluster-level examples on each. Jessica Williams, of the Volunteer Sequence Validation Corps (VSVC), provided detailed reviews of pre-0.7 outputs through January 2026 that surfaced the merge-driven ambiguity-injection bug fixed by raising `--merge-min-size-ratio` in v0.7, the homopolymer-interface question, and a partial-overlap chimera that informed the v0.6.1 fix. We are also grateful to the broader community of sequence validators across the fungal-barcoding world: every recurring question in this paper traces back to a real specimen surfaced by a real validator, and the 0.8.0 pipeline is in large measure a response to their patience.

All sequencing data analyzed in this paper were produced by Mycota Lab (Hoosier Mushroom Society) under the protocol of Russell [5]. The author thanks Stephen D. Russell for sharing the run-level outputs that made the empirical sections of this paper possible.

Substantial portions of the text were drafted with the assistance of Claude (Anthropic, Opus 4.7). The core ideas, design decisions, and empirical analyses were provided by the author; the AI contributed to exposition, LaTeX formatting, and iterative drafting under human direction. The author takes full responsibility for the content.

References

[1] J. Walker. Critical error rate analysis for MSA-based variant phasing in nanopore amplicon sequencing. Technical report, 2026.

[2] J. Walker. Homopolymer error rate analysis for nanopore amplicon sequencing. Technical report, 2026.

[3] J. Walker. Critical error rate in practice: context-aware variant significance for nanopore amplicon pipelines. Technical report, 2026.

[4] J. Walker. Speconsense: high-quality clustering and consensus generation for Oxford Nanopore amplicon reads. `https://github.com/joshuaowalker/speconsense`, 2024.

[5] S. D. Russell. ONT DNA barcoding fungal amplicons w/ MinION & Flongle V.3. *protocols.io*, March 2023. `https://dx.doi.org/10.17504/protocols.io.36wgq7qykvk5/v3`.

[6] K. Sahlin, M. Lim, and S. Prost. NGSpeciesID: DNA barcode and amplicon consensus generation from long-read sequencing data. *Ecology and Evolution*, 11(3):1392–1398, 2021.

[7] K. Sahlin and P. Medvedev. De Novo clustering of long-read transcriptome data using a greedy, quality-value based algorithm. In *Research in Computational Molecular Biology (RECOMB 2019)*, Lecture Notes in Computer Science, pages 227–242. Springer, 2019.

[8] S. van Dongen. *Graph Clustering by Flow Simulation*. PhD thesis, University of Utrecht, 2000.

[9] C. Lee. Generating consensus sequences from partial order multiple sequence alignment graphs. *Bioinformatics*, 19(8):999–1008, 2003.

[10] R. Vaser, I. Sović, N. Nagarajan, and M. Šikić. Fast and accurate de novo genome assembly from long uncorrected reads. *Genome Research*, 27(5):737–746, 2017.

[11] M. Šošić and M. Šikić. Edlib: a C/C++ library for fast, exact sequence alignment using edit distance. *Bioinformatics*, 33(9):1394–1395, 2017.

[12] T. Rognes, T. Flouri, B. Nichols, C. Quince, and F. Mahé. VSEARCH: a versatile open source tool for metagenomics. *PeerJ*, 4:e2584, 2016.

[13] J. Walker. `adjusted-identity`: a Python package for homopolymer-aware sequence identity scoring. `https://github.com/joshuaowalker/adjusted-identity`, 2025.